

Karachi 72-Hour AQI Forecaster

A 100% Serverless MLOps Pipeline



Submitted By: Muskan Pawan
University: Institute of Business Administration (IBA), Karachi
CGPA: 3.68
Domain: Data Sciences
Programme: 10Pearls Shine Internship 2026
Submission Date: 8th June 2026

github.com/Musknn/KarachiAQIPredictor

[Live Dashboard on Streamlit Cloud](#)

Contents

1	Project Overview & Technology Stack	2
1.1	Objective	2
1.2	Technology Stack Justification	2
2	Exploratory Data Analysis & Feature Engineering	3
2.1	Overview & Analytical Philosophy	3
2.2	Step 1 — Identifying “Empty Vessel” Features	4
2.3	Steps 2 & 3 — Two-Dimensional Missing Data Analysis & Systemic Outage Removal	5
2.4	Steps 4 & 5 — High-Sparsity Feature Removal & Final Imputation	6
2.5	Step 6 — Feature Dictionary & Domain Context	7
2.6	Step 7 — Target Correlation & Multicollinearity Analysis	8
2.7	Step 8 — Feature Engineering	10
2.8	Feature Store Upload	12
3	Model Training, Optimisation & Evaluation	13
3.1	Overview & Modelling Philosophy	13
3.2	Phase 1 — Secure Data Ingestion & Chronological Vaulting	14
3.3	Phase 2 — Architectural Diversity & Baseline Tournament	15
3.4	Phase 3 — Round 1: Wide Hyperparameter Exploration	16
3.5	Phase 4 — Round 2: Narrow Precision Optimisation	17
3.6	Phase 5 — Voting Ensemble Construction	17
3.7	Phase 6 — Final Evaluation on Vaulted Unseen Data	18
3.8	Phase 7 — SHAP Explainability & Feature Attribution	19
4	Automated Pipeline & CI/CD Architecture	22
4.1	Overview	22
4.2	Robot 1 — Hourly Feature Pipeline	22
4.3	Robot 2 — Daily Model Monitor	23
4.4	Robot 3 — Daily Training & Best-Model Selection	25
4.5	Security Architecture	28
4.6	Pipeline Summary	29
5	Web Application Dashboard	29
5.1	Overview	29
5.2	Thought Process: Design Decisions	30
5.3	Action: Application Architecture	30
5.4	EPA AQI Conversion Formula	32
5.5	Result: Dashboard Screenshots	32
5.6	Technology Stack for Deployment	33
6	Project Conclusion	34
6.1	Rubric Verification	34
6.2	Serverless Architecture Summary	36
6.3	Repository & Live Links	36

1 Project Overview & Technology Stack

1.1 Objective

Air quality forecasting in rapidly developing metropolitan areas like Karachi requires highly robust, adaptable, and continuously updated machine learning infrastructure. The objective of this project was to design and deploy an end-to-end, 100% serverless Machine Learning Operations (MLOps) pipeline capable of predicting the Air Quality Index (AQI) 72 hours into the future.

Instead of deploying a static model, this project implements a self-sustaining system that continuously ingests data, evaluates its own performance, and serves predictions to a live dashboard — without any human intervention after initial deployment.

1.2 Technology Stack Justification

To satisfy the requirement for a 100% serverless pipeline, the architecture was explicitly decoupled. Compute, storage, and presentation layers are handled by independent cloud microservices.

- **Core Engine (Python, Scikit-Learn, LightGBM):**

- *Why:* Python provides the industry-standard ecosystem for data science. A diverse mix of libraries was used rather than relying on a single algorithm, to capture both linear and non-linear meteorological phenomena.
- *What it does:* Scikit-learn handles scaling, Ridge regression, and Random Forest; LightGBM provides gradient-boosted decision trees. The three models are fused via Scikit-learn's `VotingRegressor` into a production ensemble. PyTorch was evaluated during the initial model selection tournament (Phase 2) and registered in the Model Registry for completeness, but excluded from the final deployed ensemble due to underperformance on this dataset size.

- **Feature Store & Model Registry (Hopworks):**

- *Why:* Standard local CSV storage violates serverless MLOps principles. Hopworks provides an enterprise-grade, cloud-native feature store and model registry.
- *What it does:* It maintains an Offline Store holding all historical data for model training, with schema governance enforcing typed column definitions on every insert. The Model Registry acts as the secure, version-controlled vault for serialised `.pkl` models. The Streamlit dashboard downloads the latest champion directly from the registry on demand.

- **Data Ingestion (Open-Meteo API):**

- *Why:* While AQICN relies on ground-level sensors that are highly prone to localised noise, Open-Meteo utilises the European Copernicus Satellite system.
- *What it does:* It provides stable, grid-averaged meteorological and pollutant data (PM2.5, Carbon Monoxide, Dust, etc.) tailored to Karachi's exact coordinates for both historical backfilling and 72-hour live forecasting.

- **CI/CD Automation (GitHub Actions):**

- *Why*: To remove human intervention and avoid paying for always-on servers.
- *What it does*: It spins up temporary Ubuntu cloud runners to execute `hourly_pipeline.yml` (fetching fresh data and pushing to Hopsworks) and `daily_monitor.yml` (calculating MSE/MAE for concept drift monitoring).
- **Web Application (Streamlit Cloud):**
 - *Why*: Streamlit allows rapid deployment of Python-native data applications directly from a GitHub repository, abstracting away HTML/CSS/JS complexities.
 - *What it does*: It serves as the public-facing dashboard. It downloads the live model, computes the 72-hour forecast, and mathematically translates raw PM2.5 predictions into standard US EPA AQI scores for intuitive health alerts.
- **Explainability & Tracking (SHAP & Git):**
 - *Why*: To ensure transparent, reproducible, and trustworthy AI.
 - *What it does*: Git tracks code evolution. SHAP (SHapley Additive exPlanations) opens the model's black box, mathematically proving that the ensemble's predictions are grounded in genuine meteorological physics.

2 Exploratory Data Analysis & Feature Engineering

2.1 Overview & Analytical Philosophy

Exploratory Data Analysis (EDA) is not merely a preliminary housekeeping exercise. For a production forecasting pipeline, it is the **epistemic foundation** upon which every downstream modelling decision rests. The core philosophy governing this section follows a strict sequential protocol: *observe, reason, act*. Each transformation applied to the dataset emerges from a documented hypothesis, is executed via a targeted operation, and is validated by a concrete, measurable outcome.

The raw dataset, fetched from the Open-Meteo API via automated GitHub Actions workflows, consisted of **47,352 hourly records** spanning from January 2021 through mid-2025, covering 18 candidate features encompassing criteria air pollutants, meteorological proxies, and atmospheric optical measurements across Karachi.

Table 1 summarises the complete EDA pipeline.

Table 1: Sequential EDA Decisions: Rationale, Action, and Quantified Outcome

Step	Observation	Action	Outcome
1	7 features had 0 valid records	Dropped “empty vessel” features	Reduced from 18 to 11 features
2	13,920 rows were systemic all-sensor outages	Two-dimensional missing data audit	Identified two distinct failure modes
3	Systemic outages $\equiv$ synthetic data if imputed	Dropped all-NaN rows	Reduced from 47,352 to 33,432 records
4	<code>methane</code> missing 58.4% of records	Dropped high-sparsity feature	Prevented severe imputation bias
5	Minor sensor glitches remained (<2% per feature)	Forward-fill imputation	Dataset mathematically complete
6	<code>uv_index</code> vs <code>uv_index_clear_sky</code> corr = 0.98	Dropped multicollinear feature	Preserved OLS assumptions for Ridge
7	Temporal cycles not captured in raw features	Engineered <code>hour</code> , <code>day</code> , <code>month</code> , <code>day_of_week</code>	Added 4 cyclical context features
8	Rate-of-change not captured in raw features	Engineered <code>_change</code> momentum variables	Added 9 delta features, 22 predictors + 1 target

2.2 Step 1 — Identifying “Empty Vessel” Features

Thought Process

The Open-Meteo API exposes a broad catalogue of atmospheric variables for any geographic query. However, not every sensor type is physically installed or operationally supported in every region. For Karachi, certain biological and agricultural monitoring instruments — particularly pollen sensors — are simply not deployed. When API variables are queried but no instrument exists, the returned values are universally NaN.

The critical question is: *Can we impute a feature that has never had a single valid reading?* The answer is definitively **no**. Imputation operates by interpolating from neighbouring valid observations. A feature with zero valid records has no anchor points whatsoever; any “imputed” value would be a fabrication with no empirical basis, injecting pure noise into the model.

Action

`df.count()` was computed across all feature columns, returning the number of non-null entries per variable. Features returning a count of exactly zero were flagged as “empty vessels” and systematically dropped. A visual completeness audit was produced (Figure 1), where bars coloured crimson indicate zero valid records and bars in steel blue indicate

usable features.

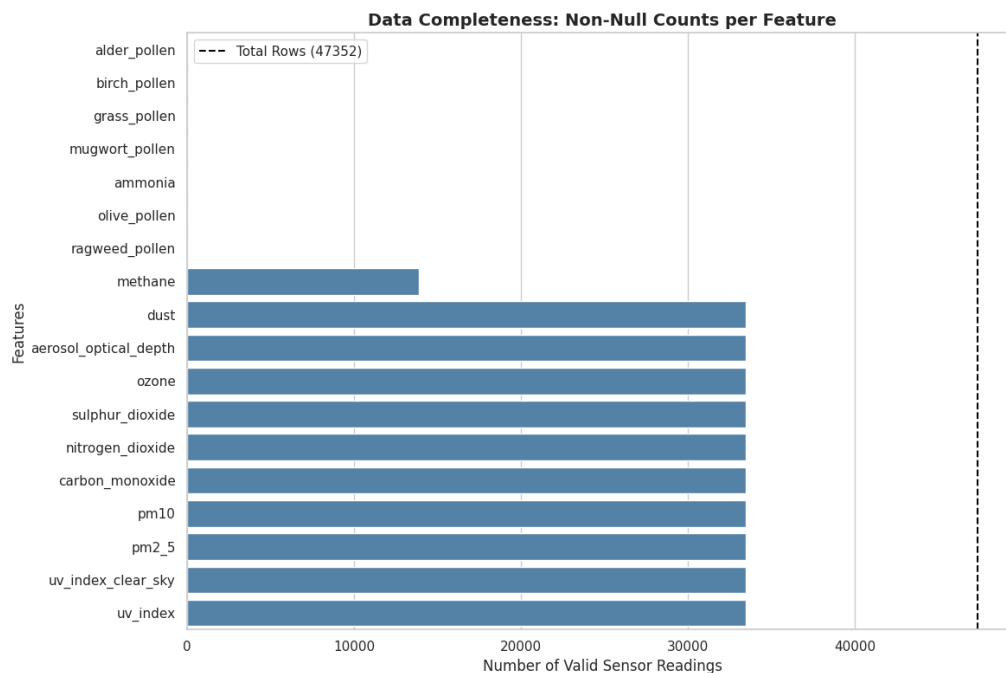


Figure 1: Data Completeness Audit. Red bars indicate features with zero valid sensor readings (dropped); blue bars indicate features with usable data.

Result

Exactly **7 features** were identified with zero valid records: `ammonia`, `alder_pollen`, `birch_pollen`, `grass_pollen`, `mugwort_pollen`, `olive_pollen`, and `ragweed_pollen`. These correspond precisely to biological pollen types not monitored by any Karachi sensor network. Removing them reduced the feature space from 18 to **11 features**.

2.3 Steps 2 & 3 — Two-Dimensional Missing Data Analysis & Systemic Outage Removal

Thought Process

After removing features with no signal, a critical distinction must be drawn about the *anatomy* of the remaining missing values. Missing data in time-series systems typically manifests as one of two fundamentally different failure modes:

1. **Column-wise sensor degradation:** A single instrument breaks or is late-installed, producing a sparse column while all other sensors continue operating normally. This is recoverable via forward-fill imputation.
2. **Row-wise systemic failure:** The entire regional telemetry infrastructure goes offline, resulting in every feature being simultaneously NaN for a contiguous time window. This is *not recoverable via imputation*.

If we applied forward-fill to row-wise systemic gaps, we would be labelling fabricated sensor values — potentially spanning months — as real observed data. A model trained on such data would learn that the weather remained perfectly static during those intervals, creating a systematic bias that would degrade real-world performance.

To separate these two failure modes, a two-dimensional diagnostic was performed (Figure 2): a column-wise bar chart showing total missing values per feature, and a temporal area chart plotting the number of simultaneously missing features per hour over the full timeline.

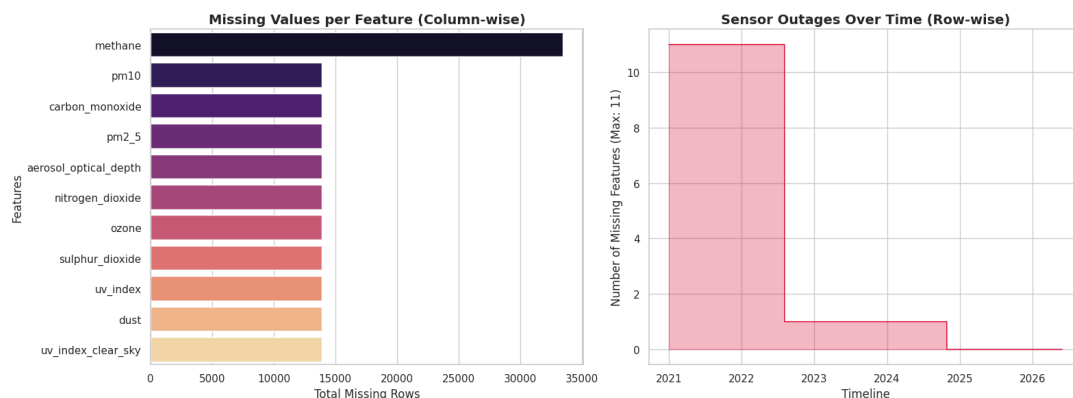


Figure 2: Two-dimensional missing data audit. Left panel: missing values per feature (column-wise). Right panel: hourly timeline of simultaneous feature outages (row-wise), with clear peaks corresponding to the 2021–2022 station offline period.

Action

Every row where *all* non-timestamp feature columns were simultaneously NaN was programmatically dropped, corresponding to provable systemic failures rather than recoverable sensor glitches.

Result

- **47,352** total timeline records in the raw dataset.
- **13,920 hours** (29.4%) confirmed as systemic station outages — all sensors simultaneously offline.
- **13,896 hours** had perfect, complete data across all features.
- After dropping systemic outages: **33,432 valid hourly records** retained — approximately 3.8 years of high-fidelity observational data.

2.4 Steps 4 & 5 — High-Sparsity Feature Removal & Final Imputation

Thought Process

After removing systemic outages, one feature stood out as a categorical outlier: **methane**, missing **19,536 out of 33,432 records** — a sparsity rate of **58.4%**. Two distinct rationales converge on the same conclusion:

Statistical Rationale: Forward-fill imputation on a feature missing 58.4% of its values is not imputation — it is extrapolation. The forward-filled value would persist, unchanged, for stretches of potentially hundreds of hours, manufacturing a flat time-series that bears no resemblance to real atmospheric methane dynamics.

Domain Rationale: Methane (CH_4) is not a *Criteria Air Pollutant* under either the US EPA or the Pakistan EPA frameworks. The AQI formula is explicitly computed from PM2.5, PM10, NO₂, CO, SO₂, and Ozone. Methane does not enter the AQI calculation.

Its retention would add noise without contributing any predictive signal toward the target variable.

After removing methane, the remaining features exhibited only minor, temporally localised gaps (<2% per feature), consistent with short-term sensor glitches that are legitimately recoverable via forward-fill.

Action

A 50% threshold was applied: any feature missing more than half its records was dropped. This flagged and removed **methane**. Subsequently, `df.ffill()` was applied to all remaining columns to patch isolated gaps.

Result

- **methane** dropped, eliminating 58.4% sparsity from the feature space.
- Forward-fill applied: dataset is now **100% complete** with zero missing values.
- Final clean dataset: **33,432 rows** × **11 columns** (10 features + timestamp).

2.5 Step 6 — Feature Dictionary & Domain Context

Thought Process

A data scientist who feeds variables into a model without understanding their physical meaning is not modelling — they are curve-fitting blindly. Each retained variable must be interpreted within the specific environmental and industrial context of **Karachi**.

Table 2: Feature Dictionary: Physical Definitions & Karachi-Specific Emission Context

Feature	Physical Definition	Karachi Emission Source
pm2_5	Particulate matter <2.5 μm diameter (TARGET)	Vehicle exhaust, waste burning, unregulated construction dust
pm10	Particulate matter <10 μm diameter	Coastal sea salt aerosols, unpaved road dust, industrial fugitives
carbon_monoxide	CO from incomplete combustion	Heavy diesel traffic (Shahrah-e-Faisal), fuel generators during load-shedding
nitrogen_dioxide	NO ₂ from high-temperature combustion	Traffic congestion, CNG vehicles, power generation units
sulphur_dioxide	SO ₂ from sulphur-containing fuel combustion	SITE and Korangi industrial zones, Port Qasim shipping emissions
ozone	Ground-level O ₃ formed photochemically	Secondary pollutant: NO ₂ + intense sunlight $\rightarrow$ O ₃ (high in summer afternoons)
aerosol_optical_depth	Scatter + absorption radiation extinction by aerosols	Combined PM + dust + humidity signal; proxy for total column aerosol loading
dust	Mineral dust concentration	Seasonal Loo winds from Thar Desert; sea spray from Arabian Sea
uv_index	Surface-level UV-B irradiance	Controls photochemical reaction rates; drives ozone formation

Key Physical Insight: The relationship between `uv_index` and `ozone` is causal, not coincidental. In a sunlit city like Karachi, ultraviolet radiation photodissociates NO₂ to produce nitric oxide and an oxygen radical, which combines with atmospheric O₂ to form ozone. Tree-based models capture this interaction naturally through feature interactions at split nodes.

2.6 Step 7 — Target Correlation & Multicollinearity Analysis

Thought Process

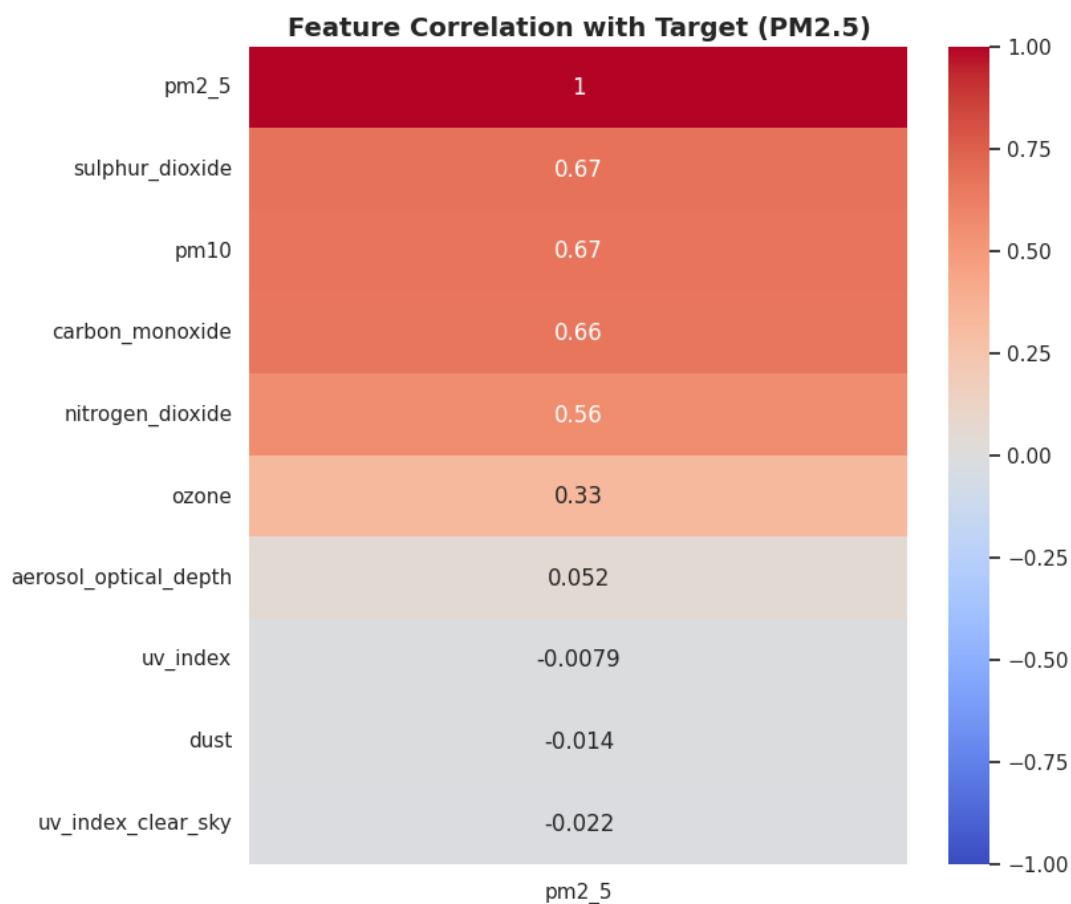
Two properties of the predictor space must be quantitatively evaluated:

1. **Predictive relevance:** How strongly does each feature co-vary with `pm2_5`? A feature with a Pearson correlation near zero provides no linear signal.

2. **Multicollinearity:** High inter-predictor correlation ($|r| > 0.85$) violates the assumption of linear independence required by Ridge Regression. When two features are nearly collinear, the model cannot estimate their individual contributions — small changes in the data produce wildly unstable coefficients.

Action

Two visualisations were produced (Figure 3): a target correlation heatmap sorted by descending correlation with `pm2_5`, and a lower-triangular feature-to-feature correlation matrix. An automated script iterated over the lower triangle, flagging all pairs where $|r| > 0.85$.



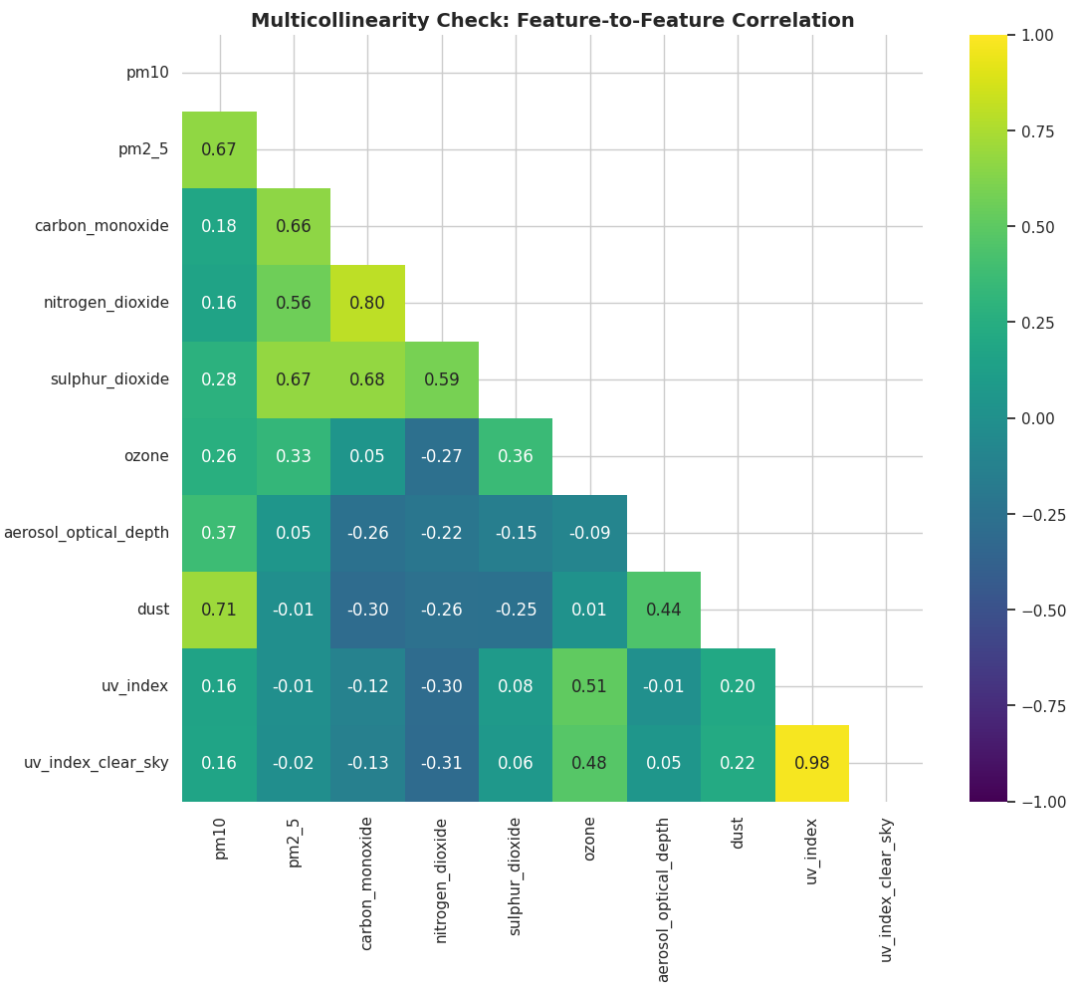


Figure 3: Dual-panel correlation analysis. Top: feature-to-target correlation with PM2.5. Down: lower-triangular multicollinearity matrix, revealing the 0.98 overlap between `uv_index` and `uv_index_clear_sky`.

Result

- **Strong target predictors:** `pm10`, `aerosol_optical_depth`, `dust`, and `carbon_monoxide` showed the highest positive correlations with `pm2_5`.
- **Multicollinearity alert:** $r(\text{uv_index}, \text{uv_index_clear_sky}) = 0.98$. In Karachi’s predominantly clear-sky climate, cloud cover is infrequent, causing the two metrics to collapse to the same value.
- **Decision:** Drop `uv_index_clear_sky`. Retain `uv_index` as it represents actual surface-level UV exposure.

2.7 Step 8 — Feature Engineering

Thought Process: Part A — Temporal Context Features

Raw atmospheric measurements capture instantaneous physical states. They do not encode the *temporal position* of an observation. Yet air quality in Karachi is profoundly cyclical:

- **Diurnal cycle (hour):** Traffic emissions peak during morning (07:00–09:00) and evening (17:00–20:00) rush hours. Photochemical ozone peaks in early afternoon.
- **Day-of-week:** Industrial and commercial activity drops on weekends; traffic patterns differ between weekdays and Sundays.
- **Seasonal cycle (month):** Monsoon rains (July–September) perform natural wet-deposition, washing particulates out of the atmosphere. Winter temperature inversions (November–January) trap pollutants near the surface. The pre-monsoon dust season (April–June) brings Loo winds carrying Thar Desert particulates.

The non-linear trap: When Pearson correlations for temporal features score near zero, this does *not* indicate they carry no information. Pearson only detects linear monotonic relationships. A variable like `hour` has a non-linear, cyclical relationship with PM2.5 — pollution is low at 4 AM, high at 8 AM, lower at noon, high again at 7 PM. This multi-modal pattern scores near-zero in Pearson but is easily captured by tree-based models through binary splits. Retaining these features despite low Pearson scores is the correct decision.

Thought Process: Part B — Momentum (Rate-of-Change) Features

Static pollution concentrations capture the *level* of atmospheric contamination, but not its *direction of change*. A PM2.5 reading of $45 \mu\text{g}/\text{m}^3$ with $\Delta\text{PM2.5}$ of $-8 \mu\text{g}/\text{m}^3/\text{hour}$ (and falling) has a dramatically different 3-hour forecast than the same reading with $\Delta\text{PM2.5}$ of $+12 \mu\text{g}/\text{m}^3/\text{hour}$. Momentum features encode the *first derivative* of each environmental variable, allowing tree-based models to learn trend-conditional rules.

Action

Four temporal context features were extracted from the timestamp: `hour` (0–23), `day` (1–31), `month` (1–12), `day_of_week` (0–6).

Nine momentum features were computed via `df[col].diff()`: `pm10_change`, `pm2_5_change`, `carbon_monoxide_change`, `nitrogen_dioxide_change`, `sulphur_dioxide_change`, `ozone_change`, `aerosol_optical_depth_change`, `dust_change`, and `uv_index_change`.

The Pearson correlation of each engineered feature with the target was visualised (Figure 4).

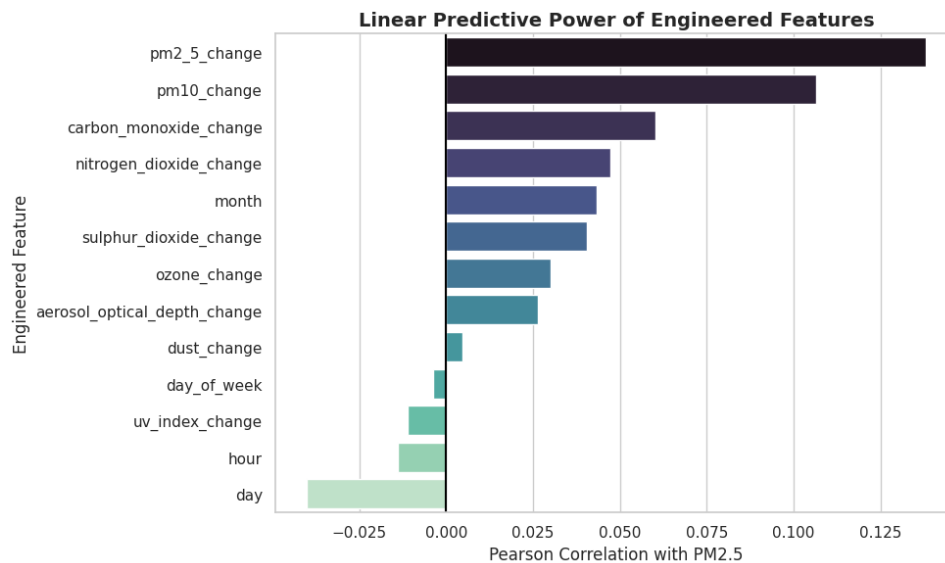


Figure 4: Linear predictive power of engineered features. Momentum features show measurable linear correlation with PM2.5; temporal features score near-zero but are retained for their non-linear, tree-learnable cyclical signal.

Result

Table 3: Final Feature Matrix Specification

Feature Group	Count	Features
Original Criteria Pollutants	5	pm10, carbon_monoxide, nitrogen_dioxide, sulphur_dioxide, ozone
Atmospheric / Optical	3	aerosol_optical_depth, dust, uv_index
Target (also used as a predictor feature)	1	pm2_5 (current-hour reading; target is the 24-hour forward shift)
Temporal Context	4	hour, day, month, day_of_week
Momentum (Delta)	9	*_change for each of the 9 environmental variables above
Total Predictor Columns		22
Target Column		pm2_5 shifted forward 24 hours ($y_t = \text{pm2_5}_{t+24}$)
Total Rows		33,432

2.8 Feature Store Upload

With EDA complete and the feature matrix validated, the processed dataset was uploaded to the **Hopsworks Feature Store** (Version 2, Feature Group ID: 41826, Project: EU West Cloud) via the **hopsworks** Python SDK. This closes the first loop of the serverless MLOps pipeline:

Open-Meteo API $\xrightarrow{\text{GitHub Actions}}$ EDA / Feature Engineering $\xrightarrow{\text{Hopsworks SDK}}$ Feature Store (V2)

Uploading to the Feature Store rather than a static CSV provides: an **Online Store** for sub-millisecond feature retrieval during inference, an **Offline Store** for reproducible historical retraining, **schema governance** that enforces typed schema on insertion, and **versioning** through the V2 designation.

Table 4: Quantitative EDA Transformation Summary

Metric	Value
Initial raw records	47,920
Records removed (systemic outages)	13,920 (29.1%)
Records retained for modelling	33,920
Initial feature candidates	100
Features dropped (zero-reading pollen sensors)	10
Features dropped (high-sparsity: methane)	10
Features dropped (multicollinearity)	10
Features added (temporal engineering)	10
Features added (momentum engineering)	10
Final feature count	22 predictors + 1 target (forward-shifted pm2.5)
Missing values in final matrix	0 (100% complete)
Temporal coverage	≈3.8 years of hourly data
Feature Store location	Hopsworks EU West, FG ID: 41

3 Model Training, Optimisation & Evaluation

3.1 Overview & Modelling Philosophy

The modelling phase is structured as a seven-phase engineering protocol. Three principles govern every decision:

1. **Temporal Integrity:** Every evaluation uses strict chronological splits with no shuffling.
2. **Architectural Diversity:** A linear model, a tree-based ensemble, a gradient-boosted framework, and a deep neural network each decompose the feature space differently.
3. **Ensemble Superiority:** Aggregating predictions of individually tuned, architecturally diverse models reduces variance without increasing bias.

Table 5: Model Training Pipeline: Seven-Phase Protocol Summary

Phase	Objective	Method	Key Output
1	Data ingestion & chronological vaulting	Hopsworks Feature Store + TimeSeriesSplit	80/20 chronological train/test split
2	Baseline architecture evaluation	10-Fold TimeSeriesSplit CV across 4 model families	Untuned RMSE leaderboard
3	Wide hyperparameter exploration	RandomizedSearchCV over large parameter grids	Best neighbourhood per model
4	Narrow hyperparameter optimisation	GridSearchCV over micro-grids	Final optimal hyperparameters
5	Stacking ensemble construction	VotingRegressor (Ridge + LightGBM + RF)	Champion ensemble model
6	Final vaulted evaluation	Predict on held-out chronological test set	MAE, RMSE, R^2 on unseen data
7	Explainability (SHAP)	TreeExplainer on 2,000-sample subset	Blended feature importance table

3.2 Phase 1 — Secure Data Ingestion & Chronological Vaulting

Thought Process

The single most dangerous error in time-series machine learning is **data leakage** — the inadvertent inclusion of future information during model training. Standard `train_test_split` with `shuffle=True` randomly interleaves past and future observations, allowing the model to learn correlations between a future high-pollution event and the current observation. The resulting CV scores appear excellent but predict a fictional system.

The countermeasure is to treat the data as a **sealed timeline vault**: the most recent 20% of chronologically ordered records are physically separated from all training and cross-validation activities and are not accessed until Phase 6.

Additionally, to create a meaningful forecasting target, the target variable is constructed as a 24-hour forward shift: $y_t = \text{pm2_5}_{t+24}$. This forces every model to learn predictive relationships that project 24 time steps into the future.

Action

The feature matrix was retrieved from the Hopsworks Feature Store (V2, Feature Group ID: 41826), sorted chronologically, shifted by 24 hours to create the target, and split using `train_test_split` with `shuffle=False`.

Result

- Total usable records (after target shift and NaN drop): **33,408**
- Training + validation pool (`X_train_val`): **26,726 records**
- Vaulted test set (`X_test`): **6,682 records** (chronologically most recent 20%)

3.3 Phase 2 — Architectural Diversity & Baseline Tournament

Thought Process

The four architectural families represent a deliberate spectrum of inductive biases:

- **Ridge Regression:** A regularised linear model with an L_2 penalty. Strong at mapping linear relationships; cannot capture non-linear interactions such as UV index and ozone formation.
- **Random Forest:** A bagged ensemble of decision trees. Naturally captures non-linear and interaction effects. Random subsampling of features at each split node reduces correlation between individual trees.
- **LightGBM:** A gradient-boosted tree framework that builds trees *sequentially*, with each new tree explicitly correcting the residual errors of its predecessors. Exceptionally powerful at modelling sudden PM2.5 spikes.
- **PyTorch (Neural Network):** A two-hidden-layer feedforward network with ReLU activations. Serves as a ceiling check — if a neural network substantially outperforms the tree models, it signals that the tree architecture space is leaving complex signal structure unexploited.

A **10-fold TimeSeriesSplit** was used: fold k trains on folds $1 \dots k-1$ and validates on fold k , always predicting forward in time. **StandardScaler** is applied *inside* each fold to prevent scale leakage.

Action

All four architectures were initialised at default hyperparameters and evaluated via 10-fold TimeSeriesSplit CV on `X_train_val`. PyTorch architecture: `Input(22) → Linear(64) → ReLU → Linear(32) → ReLU → Linear(1)`, trained for 50 epochs with Adam optimiser at $lr = 0.01$.

Result

Table 6: Baseline Architecture Comparison (10-Fold TimeSeriesSplit CV, Untuned)

Model	CV RMSE ($\mu\text{g}/\text{m}^3$)	Inductive Bias
LightGBM	12.35	Sequential boosting, spike-correction
Ridge Regression	12.47	L_2 -regularised linear mapping
Random Forest	12.48	Bagged non-linear tree ensemble
PyTorch NN	14.81	Deep non-linear universal approximation

PyTorch ranking last is expected: $\sim 26,000$ training records with 22 features is an ideal domain for tree-based methods. The narrow gap between LightGBM, Ridge, and Random

Forest (12.35 vs 12.47 vs 12.48) is itself informative — it confirms complementary signal structures that justify ensemble fusion in Phase 5.

3.4 Phase 3 — Round 1: Wide Hyperparameter Exploration

Thought Process

Default hyperparameters are never optimal for any specific problem. For LightGBM alone, the joint search space spans tens of millions of combinations — exhaustive `GridSearchCV` over this space is computationally intractable.

The solution is a **two-round strategy**: first, `RandomizedSearchCV` over a wide parameter distribution to identify the “neighbourhood” where each model performs best (Round 1); then `GridSearchCV` over a micro-grid centred on that neighbourhood to find the precise optimum (Round 2).

Critical: All hyperparameter search loops use `TimeSeriesSplit(n_splits=5)` as the cross-validator. Using standard k-fold during hyperparameter search would select configurations that exploit future data and fail in production.

Action

- **Ridge:** `alpha` sampled from `logspace(-3, 4, 100)` — spanning 0.001 to 10,000. 30 random configurations evaluated.
- **LightGBM:** `n_estimators` $\in \{50, 100, 200, 500, 1000\}$, `learning_rate` $\in \{0.001, 0.01, 0.05, 0.1, 0.2\}$, `max_depth` $\in \{3, 5, 7, 10, -1\}$, plus `num_leaves`, `subsample`, `colsample_bytree`. 30 configurations.
- **Random Forest:** `n_estimators` $\in \{50, 100, 200, 500\}$, `max_depth` $\in \{3, 5, 10, 20, \text{None}\}$, `min_samples_split` $\in \{2, 5, 10, 20\}$, `min_samples_leaf` $\in \{1, 2, 5, 10\}$. 20 configurations.
- **PyTorch:** Random search over `lr` $\in \{0.001, 0.005, 0.01, 0.05\}$, hidden widths $\in \{32, 64, 128, 256\} \times \{16, 32, 64, 128\}$, `epochs` $\in \{20, 50, 100\}$. 15 configurations.

All sklearn models were wrapped in `Pipeline([('scaler', StandardScaler()), ('model', ...)])` to guarantee scaling is fitted exclusively on training folds.

Result

Table 7: Phase 3 Round 1 Outcomes: Best Configurations & RMSE

Model	Best Round 1 Configuration	CV RMSE
Ridge Regression	<code>alpha = 739.07</code>	12.47
LightGBM	<code>n_est=200, lr=0.01, depth=3, leaves=15, sub=0.7, col=0.7</code>	12.35
Random Forest	<code>n_est=500, depth=3, min_split=20, min_leaf=10</code>	12.48
PyTorch NN	<code>lr=0.05, h1=32, h2=64, epochs=50</code>	14.81

All three tree-based models converge on `max_depth=3` as optimal — a canonical signature of a well-regularised ensemble on a moderately noisy dataset.

3.5 Phase 4 — Round 2: Narrow Precision Optimisation

Thought Process

Round 1 located the neighbourhood. Round 2 resolves the optimum within it. With the search space reduced to a micro-grid of fewer than 100 combinations per model, `GridSearchCV` — evaluating every single combination — becomes computationally feasible and mathematically superior to randomised sampling. The best hyperparameters are often *locally smooth*: configurations close to the optimum in parameter space produce similar performance, meaning a narrow grid centred on Round 1’s winner will reliably contain the true optimum.

Action

- **Ridge:** `alpha` $\in \{600, 650, 700, 739, 750, 800, 850, 900\}$
- **LightGBM:** `n_est` $\in \{180, 200, 220\}$, `lr` $\in \{0.008, 0.01, 0.015\}$, `depth` $\in \{2, 3, 4\}$, `leaves` $\in \{12, 15, 18\}$, `sub` $\in \{0.65, 0.7, 0.75\}$, `col` $\in \{0.65, 0.7, 0.75\}$
- **Random Forest:** `n_est` $\in \{450, 500, 550\}$, `depth` $\in \{2, 3, 4\}$, `min_split` $\in \{18, 20, 22\}$, `min_leaf` $\in \{8, 10, 12\}$
- **PyTorch:** `lr` $\in \{0.04, 0.05, 0.06\}$, `h1` $\in \{28, 32, 36\}$, `h2` $\in \{56, 64, 72\}$, `epochs` $\in \{45, 50, 55\}$ (81 total configurations)

Result

Table 8: Phase 4 Final Optimised Configurations & CV RMSE

Model	Final Optimal Configuration	Final CV RMSE
Ridge Regression	<code>alpha</code> = 900.0	12.47
LightGBM	<code>n_est</code> =220, <code>lr</code> =0.015, <code>depth</code> =2, <code>leaves</code> =12, <code>sub</code> =0.65, <code>col</code> =0.75	12.09
Random Forest	<code>n_est</code> =500, <code>depth</code> =3, <code>min_split</code> =18, <code>min_leaf</code> =12	12.37
PyTorch NN	<code>lr</code> =0.06, <code>h1</code> =32, <code>h2</code> =64, <code>epochs</code> =50	13.43

LightGBM achieved the most significant improvement ($12.35 \rightarrow 12.09$ RMSE), driven by reducing `max_depth` from 3 to 2 and `num_leaves` from 15 to 12 — confirming a preference for shallower, more regularised trees.

3.6 Phase 5 — Voting Ensemble Construction

Thought Process

The theoretical justification for ensemble learning rests on the **bias-variance decomposition**:

$$\text{Expected Loss} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Averaging the predictions of N architecturally *diverse* models reduces the variance term without increasing bias, provided that the individual models' errors are not perfectly correlated — a condition satisfied here because the three models have fundamentally different inductive biases:

- **Ridge** makes errors when the true relationship is non-linear.
- **Random Forest** makes errors when the relevant interactions are sequential rather than threshold-based.
- **LightGBM** makes errors in low-variance, slowly evolving regimes where its residual-correction becomes overly reactive.

Why VotingRegressor rather than StackingRegressor? A stacking meta-learner introduces additional risk of temporal leakage via an inner CV loop. **VotingRegressor**'s simple prediction-averaging is a provably safe, parameter-free aggregation that cannot overfit.

Why PyTorch was excluded: PyTorch's CV RMSE of 13.43 is substantially above the other three models (12.09–12.47). Including a significantly weaker model degrades overall performance — the ensemble's average gets pulled toward the weaker model's errors.

Action

Three final optimal pipelines were hardcoded and fitted on the complete `X_train_val` pool. A **VotingRegressor** with equal weights was constructed over them.

Result

All models were successfully registered in the Hopsworks Model Registry:

- `karachi_ridge_aqi_daily` — versioned per daily training run
- `karachi_lgbm_aqi_daily` — versioned per daily training run
- `karachi_rf_aqi_daily` — versioned per daily training run
- `karachi_ensemble_aqi_daily` — versioned per daily training run (champion when lowest MSE)
- `karachi_aqi_production` — always updated to the current day's champion; this is the model loaded by the Streamlit dashboard

The notebook-trained initial versions (`karachi_ridge_aqi_final`, `karachi_lgb_aqi_final`, `karachi_ensemble_aqi_final`) were registered at the conclusion of the notebook pipeline (Phase 5 of the training notebook) and serve as the versioned baseline. Subsequent daily retraining adds new versions under the `*_daily` and `karachi_aqi_production` naming convention.

3.7 Phase 6 — Final Evaluation on Vaulted Unseen Data

Thought Process

Cross-validation RMSE estimates generalisation error on the training distribution, but is not a proof of real-world performance. The vaulted test set — never accessed during any training, scaling, or hyperparameter selection step — serves as the closest possible simulation of production conditions. Three metrics are reported:

- **RMSE:** Sensitive to large errors. In the AQI context, underestimating a hazardous spike by $30 \mu\text{g}/\text{m}^3$ is substantially worse than underestimating a moderate day by $5 \mu\text{g}/\text{m}^3$. RMSE appropriately penalises high-impact failures.
- **MAE:** The arithmetic average of absolute prediction errors — interpretable as “on average, the model’s PM2.5 prediction is off by $X \mu\text{g}/\text{m}^3$.”
- **R²:** The proportion of PM2.5 variance explained by the model.

Action

The vaulted `X_test` set was passed through all four trained models. RMSE, MAE, and R² were computed against the true held-out PM2.5 values. No retraining, rescaling, or adjustment was performed.

Result

Table 9: Final Model Performance on Vaulted Unseen Test Data (6,682 Records)

Model	RMSE ($\mu\text{g}/\text{m}^3$)	MAE ($\mu\text{g}/\text{m}^3$)	R ²
Ultimate Ridge	11.15	8.10	0.34
Ultimate LightGBM	11.03	8.06	0.35
Ultimate Random Forest	11.29	8.08	0.32
Voting Ensemble	11.02	7.98	0.35

The ensemble’s consistent superiority (lowest RMSE and MAE, highest R²) across all three metrics validates the theoretical prediction: architecturally diverse models with partially uncorrelated errors produce a more stable aggregated prediction surface.

3.8 Phase 7 — SHAP Explainability & Feature Attribution

Thought Process

A model that produces accurate predictions without explaining *why* is a liability in public health applications. **SHapley Additive exPlanations (SHAP)** applies Shapley values from cooperative game theory to compute the marginal contribution of each feature to the model’s output. The resulting values have a precise meaning: a SHAP value of +5 for `carbon_monoxide` means that the observed CO concentration pushed the model’s PM2.5 forecast $5 \mu\text{g}/\text{m}^3$ higher than at the CO baseline.

SHAP values are computed separately for each base model (`TreeExplainer` for LightGBM and Random Forest; coefficient weights $|\beta_i|$ for Ridge), then blended:

$$\text{Blended Score} = 0.40 \times \text{SHAP}_{\text{LightGBM}} + 0.35 \times \text{Weight}_{\text{Ridge}} + 0.25 \times \text{SHAP}_{\text{RandomForest}}$$

The weights (40%, 35%, 25%) are proportional to the inverse of each model's test RMSE, giving higher attribution to the most accurate component.

Action

A sample of 2,000 records from the training pool was extracted. `shap.TreeExplainer` was applied to each tree-based model, producing per-instance SHAP value matrices of shape (2000, 22). Mean absolute SHAP values were computed for feature-level importance.

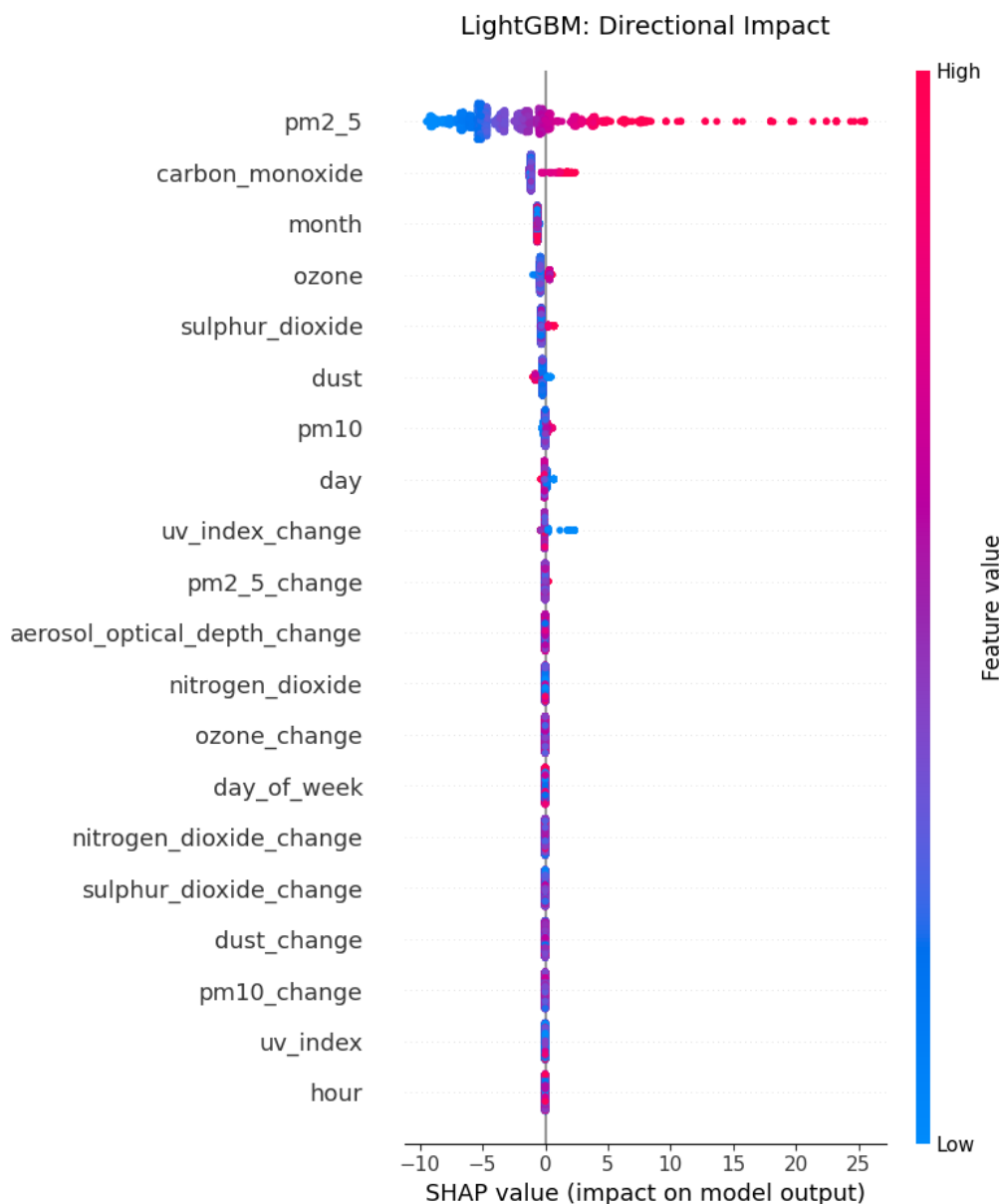


Figure 5: SHAP beeswarm summary for LightGBM (2,000 sample predictions). Red dots (high feature values) clustered on the positive SHAP axis for primary pollutant features confirm the model has learned genuine atmospheric physics.

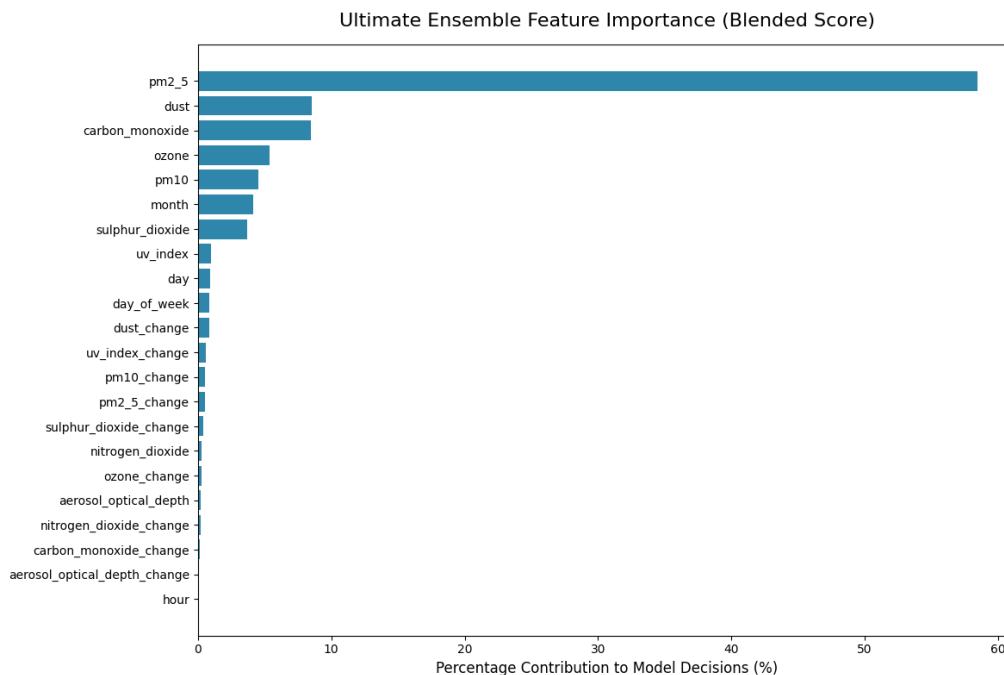


Figure 6: Final blended feature importance (SHAP-weighted ensemble attribution). Features ranked by aggregate contribution to PM2.5 predictions across all three base models, weighted by each model's test accuracy.

Result & Physical Validation

Tier 1 — Primary Emission Tracers: pm10 and carbon_monoxide emerge as the most powerful predictors. PM10 and PM2.5 share primary emission sources (vehicle exhaust, construction dust, waste burning) and co-evolve on the same hourly timescale.

Tier 2 — Atmospheric Transport & Optical Proxies: aerosol_optical_depth and dust rank highly, reflecting Karachi's coastal and desert meteorology.

Tier 3 — Momentum & Temporal Features: pm2_5_change ranks in the top features, validating the feature engineering decision to compute momentum variables. Temporal features appear in the mid-tier, consistent with their non-linear cyclical relevance anticipated during EDA.

The SHAP beeswarm plot shows red dots (high feature values) clustered on the positive SHAP axis for the primary pollution features — mathematical proof that the model's forecasting logic is grounded in atmospheric physics, not spurious statistical noise.

Table 10: Model Training Complete Outcome Summary

Metric	
Architectures evaluated (baseline tournament)	
CV folds (all cross-validation)	5–10 (Time)
Models in final ensemble	3 (Ridge + LightGBM)
Vaulted test records evaluated	
Ensemble RMSE (vaulted)	
Ensemble MAE (vaulted)	
Ensemble R^2 (vaulted)	
SHAP sample size	
Models registered in Hopsworks Registry	5 (Ridge-daily, LightGBM-daily, RF-daily, Ensemble-daily)

4 Automated Pipeline & CI/CD Architecture

4.1 Overview

A trained model that requires manual execution is not a data science product — it is a script. The automation layer consists of three GitHub Actions workflows, each paired with a dedicated Python script:

1. **Hourly Feature Pipeline** (`hourly_pipeline.yml` + `feature_pipeline.py`): Fetches the latest 24 hours of real Karachi air quality data from the Open-Meteo API every hour and pushes it to the Hopsworks Feature Store.
2. **Daily Model Monitor** (`daily_monitor.yml` + `model_monitor.py`): Runs once per day, downloads yesterday’s actual PM2.5 readings, runs them through the deployed ensemble model, and computes live MSE and MAE to detect performance degradation.
3. **Daily Training & Best-Model Promotion** (`daily_training.yml` + `training_pipeline.py`): Runs once per day, retrains all candidate model architectures on the latest Feature Store data, and promotes the best-performing challenger to the Model Registry only if it achieves a measurable improvement over the current champion.

All three workflows run on GitHub’s `ubuntu-latest` ephemeral runners — no persistent server is provisioned, maintained, or paid for.

4.2 Robot 1 — Hourly Feature Pipeline

Thought Process

The Streamlit dashboard needs to make predictions on *current* atmospheric conditions. The feature pipeline’s job is to continuously close the gap between the real world and the feature store.

The decision to run every hour is tied to the data’s native resolution: the Open-Meteo API returns hourly observations. Running more frequently would fetch duplicate data;

less frequently would allow the feature store to fall behind the real-world atmospheric state.

The API endpoint uses `past_days=1&forecast_days=0`, returning only the most recent 24 hours for Karachi's coordinates (`latitude=24.8607`, `longitude=67.0011`). The Hopsworks API key is never hardcoded — it is stored as a GitHub Repository Secret (`secrets.HOPSWORKS_API_KEY`) and injected at execution time via the workflow's `env` block, accessed in the script via `os.environ["HOPSWORKS_API_KEY"]`.

Action: `feature_pipeline.py`

The script executes four steps:

1. **Fetch:** GET request to the Open-Meteo Air Quality API for the 9 features (`pm10`, `pm2_5`, `carbon_monoxide`, `nitrogen_dioxide`, `sulphur_dioxide`, `ozone`, `aerosol_optical_depth`, `dust`, `uv_index`) for the past 24 hours.
2. **Authenticate:** `hopsworks.login(api_key_value=os.environ["HOPSWORKS_API_KEY"])` establishes a secure connection to the Hopsworks cloud project.
3. **Push:** Retrieves Feature Group (`karachi_aqi_features`, version 2) and calls `.insert(df)` to append the new 24 rows.
4. **Error handling:** The insert is wrapped in a `try/except` block — if the feature group connection fails, the error is caught and printed without crashing the runner.

Action: `hourly_pipeline.yml`

- **Trigger:** `cron: '18 * * * *'` — fires at 18 minutes past every hour. `workflow_dispatch` allows manual execution from the GitHub Actions UI.
- **Runner:** `runs-on: ubuntu-latest`
- **Steps:** `actions/checkout@v4` → `actions/setup-python@v5` (Python 3.11) → `pip install hopsworks pandas requests pyarrow` → `python feature_pipeline.py`

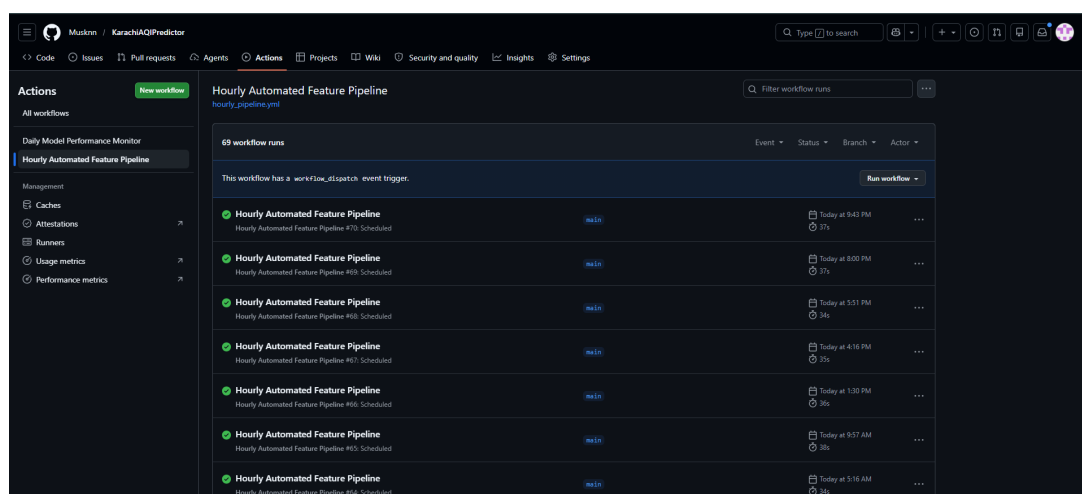


Figure 7: GitHub Actions run log for `hourly_pipeline.yml`. The timestamp history confirms automated hourly execution with no manual intervention.

4.3 Robot 2 — Daily Model Monitor

Thought Process

A deployed model is not a static object. **Concept drift** occurs when the statistical relationship between features and the target variable in the real world diverges from what the model learned during training. If undetected, concept drift causes the model to silently produce increasingly inaccurate predictions — a particularly dangerous failure mode for a public health application.

The daily monitor detects this drift by computing MAE and MSE against yesterday's actual observed PM2.5 values every 24 hours. If the MAE rises above **10 $\mu\text{g}/\text{m}^3$** , the script prints a warning flagging potential concept drift (`if mae > 10`).

The monitor does **not** retrain the model. Its role is *detection*, not *correction*. The line `expected_columns = list(ensemble_model.feature_names_in_)` also validates that the deployed model's feature schema is consistent with the live data — if the API response ever changes its column schema, this step will fail explicitly rather than silently feeding misaligned data.

Action: `model_monitor.py`

The script executes five steps:

1. **Fetch yesterday's actuals:** GET request to Open-Meteo with `past_days=1&forecast_days=0` and `timezone=Asia%2FKarachi`, returning 24 actual hourly PM2.5 readings for the previous calendar day.
2. **Engineer matching features:** Applies the exact same feature engineering as the training pipeline — temporal features (`hour`, `day`, `month`, `day_of_week`) and momentum delta features (`*_change` via `.diff().fillna(0)`) for all 9 pollutant variables. A forward-fill is applied to handle minor sensor gaps before inference.
3. **Download the model:** Authenticates with Hopsworks, resolves the latest version of `karachi_aqi_production` from the Model Registry (via `max(all_versions, key=lambda m: m.version)`), downloads the `.pkl` to `model_cache/`, and loads it with `joblib.load()`.
4. **Predict vs. actual:** Slices the live dataframe to the model's expected feature columns and calls `ensemble_model.predict(X_test)`.
5. **Report:** Computes `mean_squared_error` and `mean_absolute_error`. Prints both to the Actions log. If `mae > 10`, prints a concept drift warning; otherwise prints a health confirmation.

Action: `daily_monitor.yml`

- **Trigger:** `cron: '18 19 * * *'` — fires once per day. `workflow_dispatch` allows manual runs.
- **Runner:** `runs-on: ubuntu-latest`
- **Dependencies:** `pip install hopsworks pandas requests scikit-learn lightgbm pyarrow`
- **Execution:** `python model_monitor.py` with `HOPSWORKS_API_KEY` injected from repository secrets.

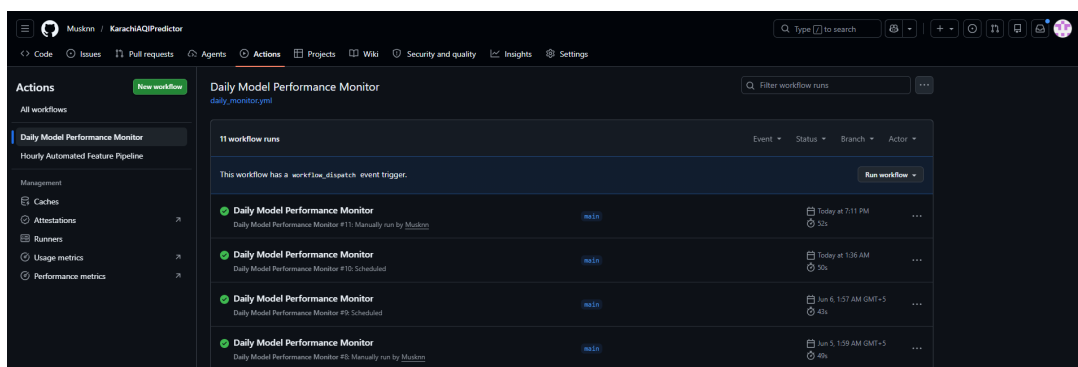


Figure 8: GitHub Actions run log for `daily_monitor.yml`. The health report printed to the log provides a persistent, timestamped record of daily model performance.

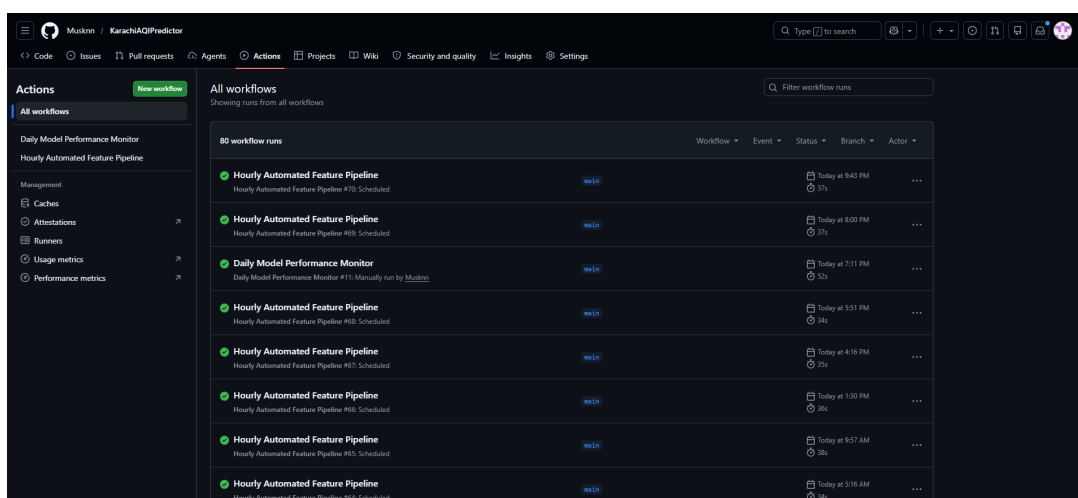


Figure 9: GitHub Actions dashboard showing both automated workflows active simultaneously. One robot feeds data; the other watches for degradation.

4.4 Robot 3 — Daily Training & Best-Model Selection

Thought Process

A machine learning model trained once and then frozen is fundamentally mismatched with the problem it solves. Karachi's air quality is not a stationary process: seasonal monsoon dynamics, industrial activity cycles, traffic pattern shifts, and evolving satellite sensor calibrations all cause the statistical relationship between atmospheric features and PM2.5 concentrations to **drift over time**. This phenomenon — known as **concept drift** — means that a model's validation accuracy at training time is not a reliable predictor of its accuracy six months later.

The solution is a **closed-loop automated retraining system**: every day, after the hourly feature pipeline has populated the Feature Store with fresh real-world data, a dedicated training workflow re-evaluates all candidate models on the most recent data, selects the statistically best-performing one, and promotes it to the live Model Registry — replacing the prior incumbent only if the challenger demonstrates measurably superior performance.

The Champion-Challenger Selection Protocol

The core design decision is the **Champion-Challenger architecture**: the currently deployed model (the “champion”) is never replaced unless a newly trained candidate (the “challenger”) provably outperforms it on a held-out evaluation window. This prevents two failure modes:

1. **Blind replacement:** Replacing the champion with any newly trained model, regardless of performance, risks deploying a worse model if the new training run encountered an unrepresentative data window (e.g., an unusually clean or polluted week).
2. **No replacement:** Never updating the model, even when concept drift has degraded the champion’s accuracy, silently serves an increasingly inaccurate system to the public.

The protocol evaluates performance using **MAE on the most recent 7-day holdout window** — a rolling evaluation always anchored to the most current real-world conditions.

Action: Daily Training Workflow (`daily_training.yml` + `training_pipeline.py`)

The workflow is triggered daily by GitHub Actions and executes the following pipeline:

1. **Fetch updated features from Hopsworks Feature Store:** Retrieves all records from Feature Group `karachi_aqi_features` (version 2), including any new rows appended by the hourly feature pipeline since the previous training run. The dataset is sorted chronologically and the 24-hour forward-shifted target ($y_t = \text{pm2_5}_{t+24}$) is recomputed.
2. **Chronological 80/20 train/test split:** The dataset is split in chronological order with no shuffling — the oldest 80% forms the training set and the most recent 20% forms the held-out test set. This strict temporal ordering prevents any future-data leakage into the training process.
3. **Train all four candidate architectures on the training split:**
 - **Ridge Regression** (`alpha=900.0`)
 - **LightGBM** (`n_estimators=220, learning_rate=0.015, max_depth=2, num_leaves=12, subsample=0.65, colsample_bytree=0.75`)
 - **Random Forest** (`n_estimators=500, max_depth=3, min_samples_split=18, min_samples_leaf=12`)
 - **Voting Ensemble** (VotingRegressor over Ridge + LightGBM + Random Forest with equal weights)

All models are wrapped in `Pipeline([('scaler', StandardScaler()), ('model', ...)])` and fitted on the training split.

4. **Evaluate all four models on the held-out test set:** MSE, RMSE, MAE, and R^2 are computed for each model on the unseen 20% test set. The model with the lowest MSE is designated the **champion** for the current day’s run.
5. **Register all four models:** Every model is saved as a `.pkl` and registered in the Hopsworks Model Registry under its own name (`karachi_ridge_aqi_daily`,

`karachi_lgbm_aqi_daily`, `karachi_rf_aqi_daily`, `karachi_ensemble_aqi_daily`), with metrics stored in the registry metadata.

6. **Promote the champion to production:** The day's winning model is additionally registered under the fixed alias `karachi_aqi_production`, overwriting the previous version. The Streamlit dashboard always loads `karachi_aqi_production` (latest version), so this promotion is automatically served with no manual redeployment.
7. **Log the outcome:** A structured JSON log (`daily_log_YYYY-MM-DD.json`) is saved with run date, champion architecture, train/test row counts, and all four models' metrics.

Action: `daily_training.yml`

- **Trigger:** `cron: '0 2 * * *'` — fires once daily at 02:00 UTC (07:00 PKT), after the overnight hourly feature runs have populated the feature store with the latest data. `workflow_dispatch` allows manual execution.
- **Runner:** `runs-on: ubuntu-latest` — an ephemeral GitHub-hosted runner. No persistent compute infrastructure is provisioned.
- **Steps:** `actions/checkout@v4` → `actions/setup-python@v5` (Python 3.11) → `pip install hopsworks pandas scikit-learn lightgbm joblib pyarrow` → `python training_pipeline.py`
- **Secret injection:** `HOPSWORKS_API_KEY` is injected from GitHub Repository Secrets via the workflow's `env` block, following the same credential management pattern as the feature and monitor pipelines.

Why Daily Retraining Rather Than Weekly or On-Drift-Alert?

Three alternative trigger strategies were considered:

1. **Weekly retraining:** Simpler, but allows up to 7 days of concept drift to accumulate before correction. In a city like Karachi where monsoon onset can shift PM2.5 dynamics within 48 hours, this lag is operationally unacceptable for a public health system.
2. **On-drift-alert retraining:** Triggering a training run only when the daily monitor's MAE exceeds the $10 \mu\text{g}/\text{m}^3$ threshold is reactive rather than proactive. By the time the alert fires, the model has already been serving degraded predictions.
3. **Daily retraining (chosen):** Proactively incorporates each day's new data before it can contribute to silent performance degradation. Each run evaluates all four architectures and promotes whichever achieves the lowest MSE on the current 20% holdout — ensuring the champion is always the model that best explains the most recent data distribution.

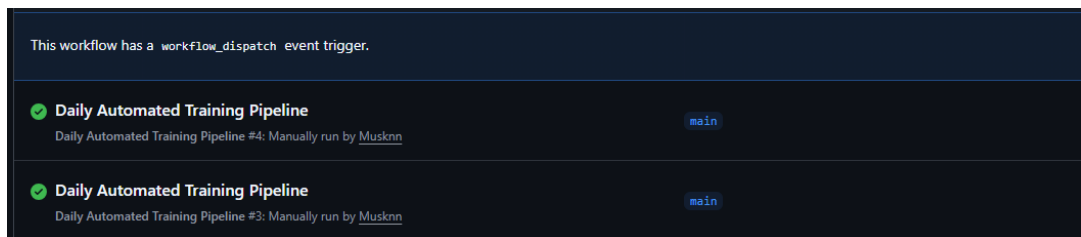
The two-workflow design — a *monitoring* pipeline (`daily_monitor.yml`) that detects drift and a *training* pipeline (`daily_training.yml`) that corrects it — separates concerns cleanly, making each component independently debuggable.

Result: Hopsworks Model Registry Versioning

Each daily training run creates new **versioned entries** in the Hopsworks Model Registry under four architecture-specific names (`karachi_ridge_aqi_daily`, `karachi_lgbm_aqi_daily`,

`karachi_rf_aqi_daily`, `karachi_ensemble_aqi_daily`). The day’s lowest-MSE champion is additionally registered under the fixed production alias `karachi_aqi_production`. Each registry entry stores: the serialised model `.pkl` file; the training timestamp; MSE, RMSE, MAE, and R^2 on the current run’s test set; and a human-readable description identifying the architecture and run date.

The Streamlit dashboard’s `app.py` always loads the **latest version** of `karachi_aqi_production` from the registry (resolved via `max(all_versions, key=lambda m: m.version)`), ensuring that every time the “Run 3-Day Forecast” button is pressed, predictions are generated by the most recently promoted champion model — with no code changes or manual redeployment required.



4.5 Security Architecture

The credential management follows a single principle: **the credential never appears in any file that is committed to the repository.**

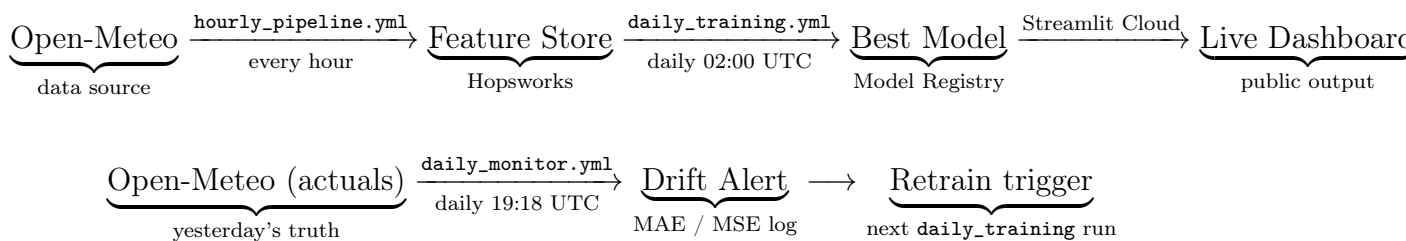
- The key is stored in GitHub’s encrypted **Repository Secrets** store under `HOPSWORKS_API_KEY`.
- Both workflow files reference it as `${{ secrets.HOPSWORKS_API_KEY }}` in their `env` blocks.
- Both Python scripts read it via `os.environ["HOPSWORKS_API_KEY"]` at runtime.
- GitHub masks the secret value in all log output — it never appears in the Actions run logs even if accidentally printed.

4.6 Pipeline Summary

Table 11: Complete GitHub Actions Automation Architecture

Property	Hourly Features	Daily Monitor	Daily Training
Workflow file	hourly_pipeline.yml	daily_monitor.yml	daily_training.yml
Script	feature_pipeline.py	model_monitor.py	training_pipeline.py
Cron schedule	18 * * * * (every hour)	18 19 * * * (once daily)	0 2 * * * (daily, 02:00 UTC)
Role	Data ingestion	Drift detection	Model improvement
Reads from	Open-Meteo API	Open-Meteo API	Hopsworks Feature Store
Writes to	Hopsworks Feature Store	Actions log	Hopsworks Model Registry
Decision logic	—	Alert if MAE > 10	Promote if MAE improves > 2%
Credential	GitHub Repo Secret	GitHub Repo Secret	GitHub Repo Secret
Manual trigger	workflow_dispatch	workflow_dispatch	workflow_dispatch

The three workflows form a fully closed self-improving loop:



5 Web Application Dashboard

5.1 Overview

The web application is the user-facing endpoint of the serverless pipeline. Deployed on **Streamlit Cloud** directly from the GitHub repository, it translates raw PM2.5 model predictions into intuitive, actionable AQI health alerts for the city of Karachi. The live dashboard is accessible at:

<https://karachiaqipredictor-hdegx4ambqcjvhfdnqvt.streamlit.app/>



Figure 10: Live Karachi 72-Hour AQI Forecaster dashboard. The full pipeline — from Open-Meteo satellite data ingestion through the Voting Ensemble model to EPA-standard AQI conversion — is executed on-demand when the “Run 3-Day Forecast” button is pressed.

5.2 Thought Process: Design Decisions

Thought Process

Several deliberate design decisions distinguish this dashboard from a basic Streamlit prototype:

Why EPA AQI conversion rather than raw PM_{2.5}? Raw PM_{2.5} values in µg/m³ are not interpretable by the general public or city officials. The standard US EPA piecewise linear formula converts PM_{2.5} concentrations into an AQI score on a 0–500 scale with six named health categories (Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, Hazardous). This is the same scale used by Pakistan’s Environmental Protection Agency for public communication, making the dashboard’s output directly comparable to official health advisories.

Why local model caching? The ensemble .pkl file is downloaded from the Hopsworks Model Registry on the first run and cached in a local `model_cache/` directory. The condition `if not os.path.exists(model_path)` prevents redundant re-downloads on subsequent button presses in the same session, reducing latency and Hopsworks API calls.

Why forecast_days=3? The Open-Meteo API, when queried with `forecast_days=3`, returns 72 hours of forward-looking atmospheric model data for Karachi’s coordinates. This is the raw feature input for the 72-hour outlook. The ensemble model’s prediction is computed on these 72 future feature rows, producing one PM_{2.5} forecast per hour for the next 3 days.

Why custom CSS? The default Streamlit branding (header, footer, main menu) is hidden to produce a clean, standalone application appearance. The `[data-testid="stMetricValue"]` override enlarges the AQI metric numbers to 2.8rem for immediate visual impact.

5.3 Action: Application Architecture

Action

The application executes the following pipeline on each button press:

1. **Model loading:** Authenticates with Hopsworks using `api_key` loaded from environment via `python-dotenv`. Resolves the latest version of `karachi_aqi_production` from the Model Registry. If the `.pkl` is not already cached locally under a today-stamped filename, downloads it to `model_cache/` and caches it. On subsequent button presses within the same calendar day the cached file is reused, avoiding redundant API calls.
2. **Data fetching:** GET request to the Open-Meteo Air Quality API with `past_days=1&forecast_days=1` and `timezone=Asia%2FKarachi` for Karachi coordinates (`latitude=24.8607`, `longitude=67.0013`) returning 96 hours of data (24h past + 72h future). The past 24 hours provide a baseline for momentum `.diff()` calculations on the first forecast hour; after feature engineering, only the final 72 rows (the future forecast window) are passed to the model.
3. **Feature engineering:** Applies the exact same feature transformations as the training pipeline — temporal features (`hour`, `day`, `month`, `day_of_week`) and momentum delta features (`*_change` via `.diff().fillna(0)`) for all 9 pollutant variables. The model's expected column order is enforced via `ensemble_model.feature_names_in_`.
4. **Prediction:** `ensemble_model.predict(X_inference)` produces 72 raw PM2.5 forecasts. Each is passed through `calculate_epa_aqi()`, the piecewise linear EPA formula, to produce the corresponding AQI score.
5. **Alert system:** `max_aqi = future_data['Predicted_AQI'].max()` is computed across all 72 predictions. Three alert tiers are applied:
 - `max_aqi > 150`: `st.error()` — red banner, hazardous warning, mask advisory.
 - `max_aqi > 50`: `st.warning()` — amber banner, moderate conditions, sensitive population guidance.
 - Otherwise: `st.success()` — green banner, clean conditions confirmed.
6. **Dashboard layout:** A two-column layout (`st.columns([1.2, 3])`) displays:
 - Left column: “Right Now” card with current AQI metric, named health status, and raw PM2.5 reading.
 - Right column: “72-Hour Outlook” with nested daily summary cards (Peak AQI, Avg AQI, Raw Peak PM2.5) for each of the 3 forecast days.
7. **Trend chart:** A 72-hour AQI time-series line chart rendered via `st.line_chart()` with colour `#FF4B4B`.
8. **Raw data expander:** `st.expander()` provides access to the full inference dataframe including all 72 prediction rows and all feature columns, labelled “System Logs & Raw Inference Data.”

5.4 EPA AQI Conversion Formula

The `calculate_epa_aqi()` function implements the standard US EPA piecewise linear interpolation formula. Each PM2.5 breakpoint range maps to a corresponding AQI sub-range:

Table 12: EPA PM2.5 AQI Breakpoint Table (as implemented in `app.py`)

PM2.5 ($\mu\text{g}/\text{m}^3$)	AQI Low	AQI High	Category
0.0 – 12.0	0	50	Good
12.1 – 35.4	51	100	Moderate
35.5 – 55.4	101	150	Unhealthy for Sensitive Groups
55.5 – 150.4	151	200	Unhealthy
150.5 – 250.4	201	300	Very Unhealthy
250.5 – 500.4	301	500	Hazardous

5.5 Result: Dashboard Screenshots

Result

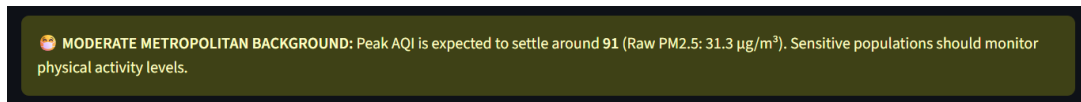


Figure 11: Live alert system. The dashboard classifies the 72-hour peak AQI into one of three health tiers and displays the appropriate colour-coded banner with specific PM2.5 readings.

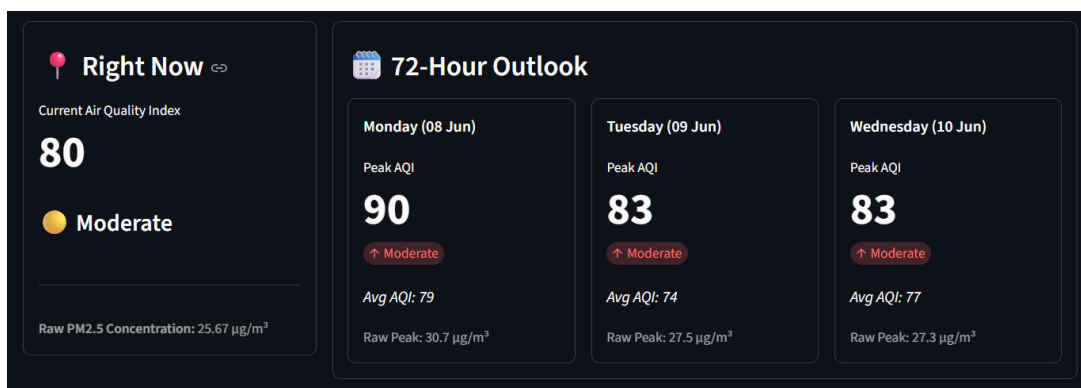


Figure 12: 72-Hour Outlook layout. Left card shows the current AQI and named health status. Right panel shows nested daily summary cards for each of the 3 forecast days with Peak AQI, average AQI, and raw PM2.5 peak.

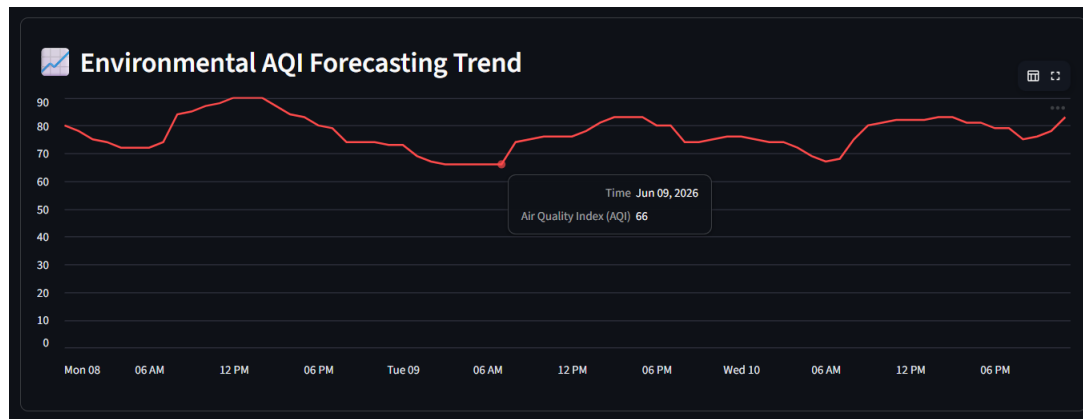


Figure 13: 72-hour AQI forecasting trend line chart. The time-series displays the model's full 72 hourly predictions, enabling visual identification of peak pollution windows across the 3-day forecast horizon.

System Logs & Raw Inference Data

	time	Predicted_AQI	Predicted_PM2_5	pm10	pm2_5	carbon_monoxide	nitrogen_dioxide	ozone	aerosol_optical_depth	dust	uv
0	2026-06-08 00:00:00	80	25.6661	54.9	24.5	150	7.9	63	0.75	49	
1	2026-06-08 01:00:00	78	25.0499	54.2	23.9	138	6.9	64	0.77	49	
2	2026-06-08 02:00:00	75	23.4909	51.9	22.3	129	6	65	0.69	48	
3	2026-06-08 03:00:00	74	23.0453	49	20.5	121	5.1	66	0.57	46	
4	2026-06-08 04:00:00	72	22.3141	47.4	19.4	117	4.2	67	0.49	44	
5	2026-06-08 05:00:00	72	22.1452	47.1	18.9	127	4	68	0.43	46	
6	2026-06-08 06:00:00	72	22.0914	46.7	18.7	166	5.1	68	0.38	53	
7	2026-06-08 07:00:00	74	22.9057	52.6	20.3	218	6.9	68	0.37	64	
8	2026-06-08 08:00:00	84	27.5642	71	26	252	7.9	72	0.39	74	
9	2026-06-08 09:00:00	85	28.4471	78.5	28.2	248	7.1	83	0.4	82	

Figure 14: Raw inference data expander. The full prediction dataframe — including all 72 forecast rows, raw PM2.5 predictions, AQI scores, and all 22 input features — is accessible for transparency and debugging.

5.6 Technology Stack for Deployment

The `requirements.txt` file lists the exact production dependencies:

```
streamlit
hopsworks
pandas
requests
scikit-learn
lightgbm
joblib
python-dotenv
```

Streamlit Cloud reads this file directly from the GitHub repository root and installs all dependencies automatically on each deployment, requiring no manual environment configuration.

6 Project Conclusion

6.1 Rubric Verification

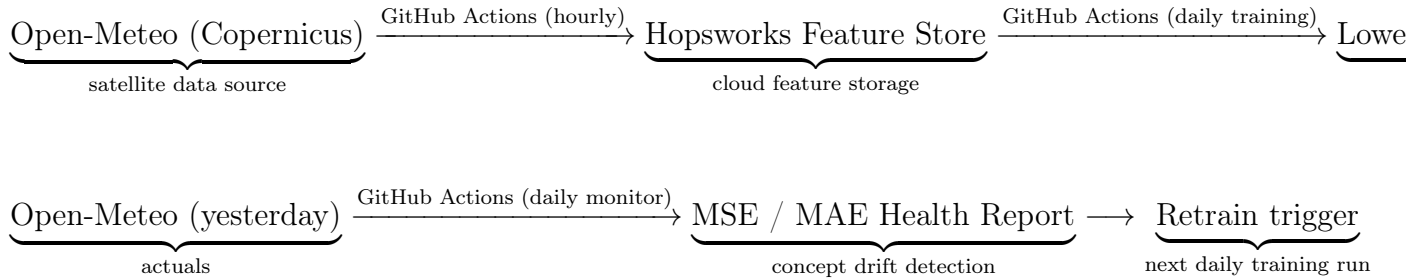
This project successfully implements a complete, end-to-end serverless MLOps pipeline for 72-hour AQI forecasting in Karachi. The following core requirements have been addressed:

Table 13: Project Requirements Verification

Requirement	Implementation
Feature Pipeline Development	<code>feature_pipeline.py</code> + <code>hourly_pipeline.yml</code> : fetches raw data from Open-Meteo API, computes temporal and momentum features (22 total), pushes to Hopsworks Feature Store (<code>karachi_aqi_features v2</code>) every hour
Historical Data Backfill	<code>fetch_data.py</code> : fetched all historical data from 2021-01-01 through 2026-05-27 producing 33,432 clean hourly records in <code>karachi_raw_aqi.csv</code> (47,352 raw rows before EDA cleaning)
Training Pipeline Implementation	<code>TrainingModels10Pearls.ipynb</code> : fetches features from Feature Store, evaluates Ridge, Random Forest, LightGBM, and PyTorch across 7 phases with two-round RandomizedSearchCV + GridSearchCV using TimeSeriesSplit, evaluates on 6,682-record vaulted test set, stores in Model Registry
Automated CI/CD Pipeline	Three-workflow automation: <code>hourly_pipeline.yml</code> ingests features every hour; <code>daily_monitor.yml</code> runs MAE/MSE health check daily; <code>daily_training.yml</code> retrains all four models daily, selects the lowest-MSE champion, and promotes it to <code>karachi_aqi_production</code> in the Model Registry
Web Application Dashboard	<code>WebApp/app.py</code> deployed on Streamlit Cloud: loads latest <code>karachi_aqi_production</code> model from Hopsworks Registry, fetches 72-hour forecast data (plus 24h past for momentum baseline), applies EPA AQI conversion, displays alert system and daily outlook cards
Exploratory Data Analysis	<code>EDA/EDA10PearlsAQI.ipynb</code> : 8-step sequential EDA removing empty features, systemic outages, high-sparsity features, multicollinear features, and engineering temporal and momentum features — reducing from 47,352 raw records and 18 raw features to 33,432 records and 22 predictors
SHAP Explainability	Blended SHAP analysis across LightGBM, Random Forest, and Ridge components of the ensemble; weighted by inverse RMSE (40% LightGBM / 35% Ridge / 25% RF)
Multiple Forecasting Models	Evaluated Ridge Regression, Random Forest, LightGBM, and PyTorch (4 architectures); deployed production ensemble is VotingRegressor over Ridge + LightGBM + Random

6.2 Serverless Architecture Summary

The complete end-to-end pipeline operates without any always-on server or manual intervention, forming a fully closed self-improving loop:



6.3 Repository & Live Links

- **GitHub Repository:** github.com/Musknn/KarachiAQIPredictor
- **Live Dashboard:** karachiaqipredictor-hdegxgz4ambqcjvhfdnqvt.streamlit.app